Programming Languages

An Introduction and Motivation

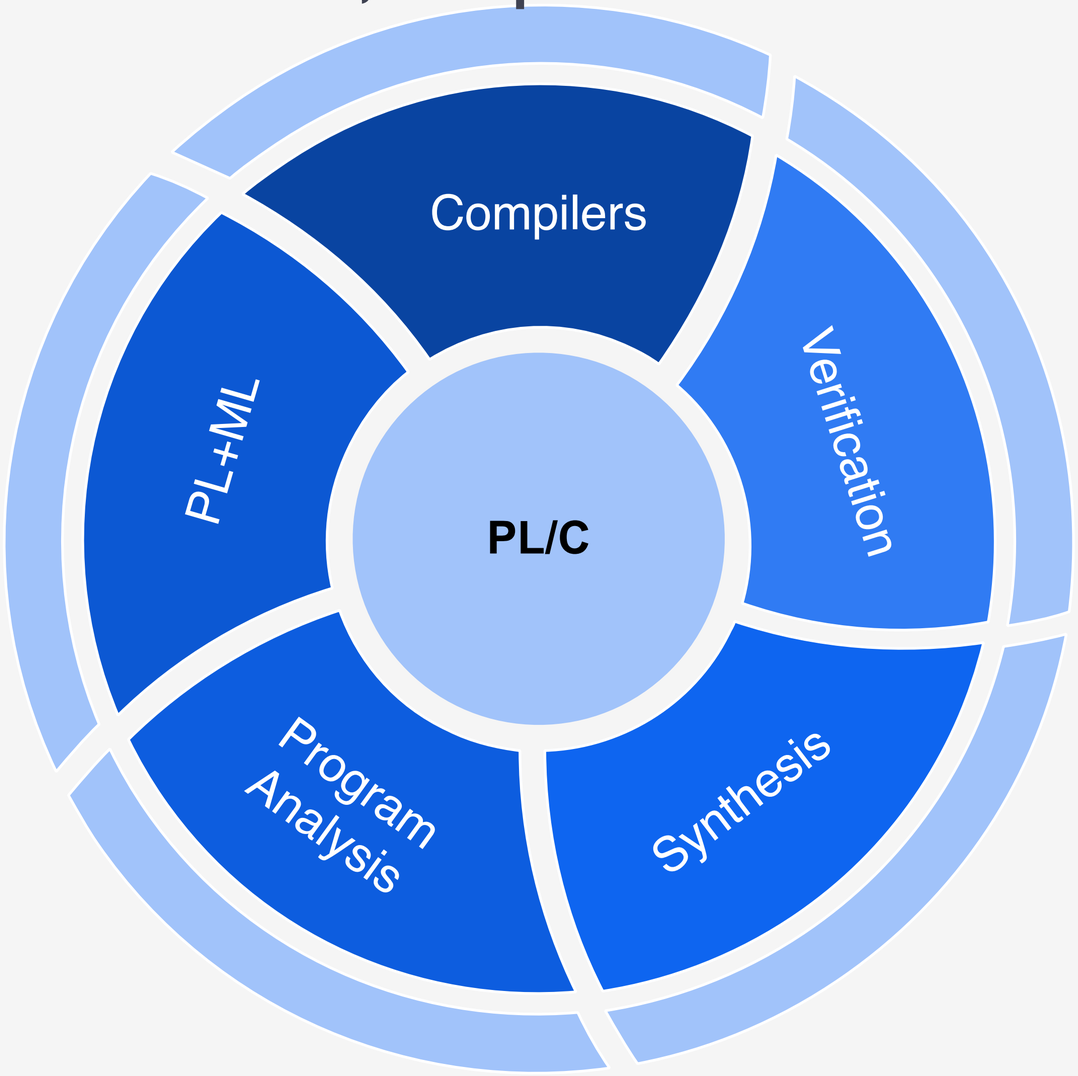
भारतीय सांकेतिक विज्ञान संस्थ हैदराबाद
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Ashish Mishra, September 25, 2025

Disclaimer!

Different sources, I thank all the sources and all Copyrights apply, wherever missed!

PL/C Sub-domains, A quick show of hands!



Programming Languages and Abstractions

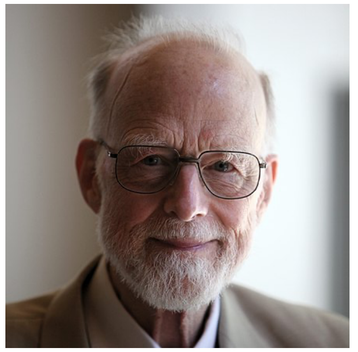
Programming Languages

- **Kids**: Languages to interact with computers.
  - Then why have so many of these.
  - There are not so many of Human Languages we learn in our life-time?
- **Adults**: Languages to solve problems.
  - C/C++ : System problems
  - Python/Lisp/Julia : Manipulating Lists/Matrices, etc.
- **Legends**: Its a tool for building Abstractions

Programming Languages

- are tools for building abstractions that enable us to reason about the behavior of the system.
  - Better or worse-suited for a particular task or a collection of tasks

Abstractions, In the words of GOATs



- Abstraction arises from a recognition of similarities between certain objects, situations, or processes in the real world, and the decision to concentrate on these similarities and to ignore for the time being the differences - C. A. R. Hoare

- The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise. - E. W. Dijkstra

Abstractions we sub-consciously use

```
1 + 2 =   3

“In” + “dia” =   “India”

{1,2} + {3,4} =   {1,2,3,4}

{1,2} + “In” =   ?, discomfort
```

Function Abstraction

```
/** Returns: a sum of the two given arguments
 *   Requires: Two element of a given type
 *
 */
T add(T x, T y) {
    ...
}
```

Can have different implementation of the *abstract* idea of addition

Abstractions are also defined for data, through Abstract Data Types.

Programming Languages

- are tools for building abstractions that enable us to reason about the behavior of the system.
- These languages must have some differences in the abstractions they define.

What abstractions are good for a task?
a.k.a: Paradigms of Programming

Programming Languages Paradigms

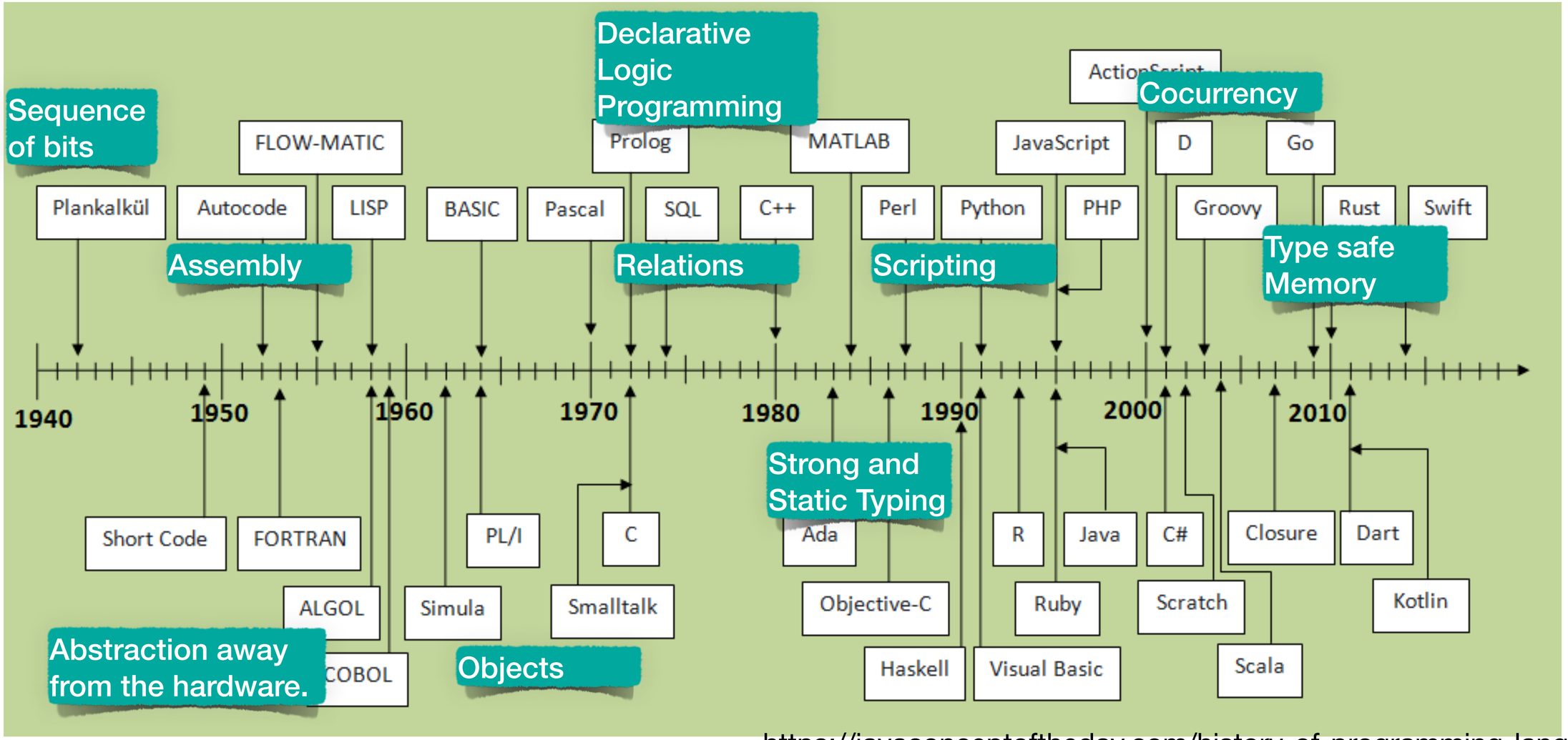
- A paradigm is the preferred approach to programming that a language supports.
- Main paradigms in scientific computation (many others for other fields):
  - Imperative.
  - Object-Oriented.
  - Functional.

A program is a list of instructions that change a memory state until the desired end state is achieved

Object receives messages, processes data, and sends messages to other objects.

All computations are implemented through functions: functions are first-class citizens

Evolution of PL is evolution of Abstractions



https://javaconceptoftheday.com/history-of-programming-languages/

Programming Languages Research

What is PL Research?

View: programming language as having a central place in solving computing problems

- Develop general abstractions, or building blocks, for solving problems, or classes of problems.
- Look for simple, understandable solutions that are also general.

The Programming Languages Enthusiast

HOME ABOUT THE PL ENTHUSIAST

← Ranking CS Departments by Publication Productivity, Interactively

The PL Enthusiast Turns One! →

BY MICHAEL HICKS | MAY 27, 2013 - 12:00 PM

| Jump to Comments

What is PL research and how is it useful?

If you are in the world of programming languages research, the announcement that UW had hired Ras Bodik away from Berkeley was big news. Quoting UW's announcement:

> Ras's arrival creates a truly world-class programming languages group in UW CSE that crosses into systems, databases, security, architecture, and other areas. Ras joins recent hires Emina Torlak, Adrian Sampson, Xi Wang, and Zach Tatlock, and senior faculty members Dan Grossman and Mike Ernst.

And there's also Luis Ceze, a regular publisher at PLDI, who ought to be considered as part of this group. With him, UW CSE has 8 out of 54 faculty with strong ties to PL. Hiring five PL-oriented faculty in three years, thus making PL a significant fraction of the faculty's expertise, is (highly) atypical. What motivated UW CSE in its decision-making? I don't know for sure, but I suspect they see that *PL-oriented researchers are making huge inroads on important problems*, bringing a useful perspective to unlock new results.

In this post, I argue why studying PL (for your PhD, Masters, or just for fun) can be interesting and rewarding, both because of what you will learn, and because of the increasing opportunities that are available, e.g., in terms of impactful research topics and funding for them.

What is PL Research?

When your hear that someone's research area is programming languages, what do you think they do?

Search

Recent Posts

- Evaluating Empirical Evaluations (for Fuzz Testing)
- Software Security is a Programming Languages Issue
- Teaching Programming Languages (part 2)
- Teaching at Scale with Clickers
- Teaching Programming Languages

Subscribe to Blog via Email

Enter your email address to subscribe to this blog and receive notifications of new posts by email.

Join 2,099 other subscribers

Email Address

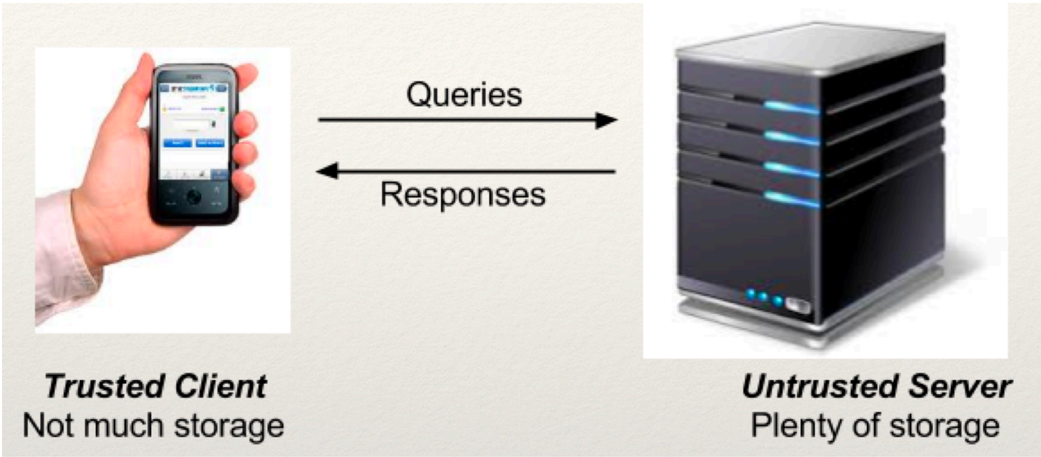
Subscribe

Recent Comments

- Matt D. on Software Security is a Programming Languages Issue
- Resumen de lecturas compartidas durante julio-de 2018 | Vonfigura on Teaching Programming Languages
- Michael Hicks on Software Security is a Programming Languages Issue
- Michael Hicks on Evaluating Empirical Evaluations (for Fuzz Testing)
- Hugo van der Sanden on Evaluating Empirical Evaluations (for Fuzz Testing)

Example of General Solutions

Authenticated Data Structures



- Merkle tree (1988): Complete tree, where server answers queries with evidence the answer is correct

- Since then, separate papers on: *sets, dictionaries, range trees, graphs, skip lists, B-trees, hash trees, ...*

General ADS Construction [POPL' 14]

Authenticated Data Structures, Generically

Andrew Miller, Michael Hicks, Jonathan Katz, and Elaine Shi
University of Maryland, College Park, USA

Abstract

An authenticated data structure (ADS) is a data structure whose operations can be carried out by an untrusted *prover*, the results of which a *verifier* can efficiently check as authentic. This is done by having the prover produce a compact proof that the verifier can check along with each operation's result. ADSs thus support outsourcing data maintenance and processing tasks to untrusted servers without loss of integrity. Past work on ADSs has focused on particular data structures (or limited classes of data structures), one at a time, often with support only for particular operations.

This paper presents a generic method, using a simple extension to a ML-like functional programming language we call λ• (lambda-auth), with which one can program authenticated operations over any data structure defined by standard type constructors, including recursive types, sums, and products. The programmer writes the data structure largely as usual and it is compiled to code to be run by the prover and verifier. Using a formalization of λ• we prove that all well-typed λ• programs result in code that is secure under the standard cryptographic assumption of collision-resistant hash functions. We have implemented λ• as an extension to the OCaml compiler, and have used it to produce authenticated versions of many interesting data structures including binary search trees, red-black+ trees, skip lists, and more. Performance experiments show that our approach is efficient, giving up little compared to the hand-optimized data structures developed previously.

Categories and Subject Descriptors D.3.3 [*Programming Languages*]: Language Constructs and Features—Data types and structures

General Terms Security, Programming Languages, Cryptography

Such a scenario can be supported using *authenticated data structures* (ADS) [5, 24, 31]. ADS computations involve two roles, the *prover* and the *verifier*. The mirror plays the role of the prover, storing the data of interest and answering queries about it. The client plays the role of the verifier, posing queries to the prover and verifying that the returned results are authentic. At any point in time, the verifier holds only a short *digest* that can be viewed as summarizing the current contents of the data; an authentic copy of the digest is provided by the data owner. When the verifier sends the prover a query, the prover computes the result and returns it along with a *proof* that the returned result is correct; both the proof and the time to produce it are linear in the time to compute the query result. The verifier can attempt to verify the proof (in time linear in the size of the proof) using its current digest, and will accept the returned result only if the proof verifies. If the verifier is also the data provider, the verifier may also update its data stored at the prover; in this case, the result is an updated digest and the proof shows that this updated digest was computed correctly. ADS computations have two properties. *Correctness* implies that when both parties execute the protocol correctly, the proofs given by the prover verify correctly and the verifier always receives the correct result. *Security*[1] implies that a computationally bounded, malicious prover cannot fool the verifier into accepting an incorrect result.

Authenticated data structures can be traced back to Merkle [18]; the well-known *Merkle hash tree* can be viewed as providing an authenticated version of a bounded-length array. More recently, authenticated versions of data structures as diverse as sets [23, 27], dictionaries [1, 12], range trees [16], graphs [13], skip lists [11, 12], B-trees [21], hash trees [26], and more [15] have been proposed. In each of these cases, the design of the data structure, the supporting operations, and how they can be proved authentic have been reconsidered from scratch, involving a new, potentially tricky proof of security. Arguably, this state of affairs has hindered the advance-

- **Simple language extension**, data structure written mostly as usual. Different code generated for client and server
  - Expresses many prior ADSs
- Proved that type correctness implies authenticity
  - One proof for all!

Elements of PL Research

- *Design*: What feature, analysis, transformation, etc.?
- *Reasoning*: What does it mean? Why is what you are doing correct?
- *Implementation*: How do you implement this language, analysis, transformation, etc.
- *Synthesizing*: Automatically generating programs from user intent.

Implementation: This is what Compilers are about

Machines don’t run our programs!

```
8B542408 83FA0077 06B80000 0000C383
FA027706 B8010000 00C353BB 01000000
B9010000 008D0419 83FA0376 078BD989
C14AEBF1 5BC3
```

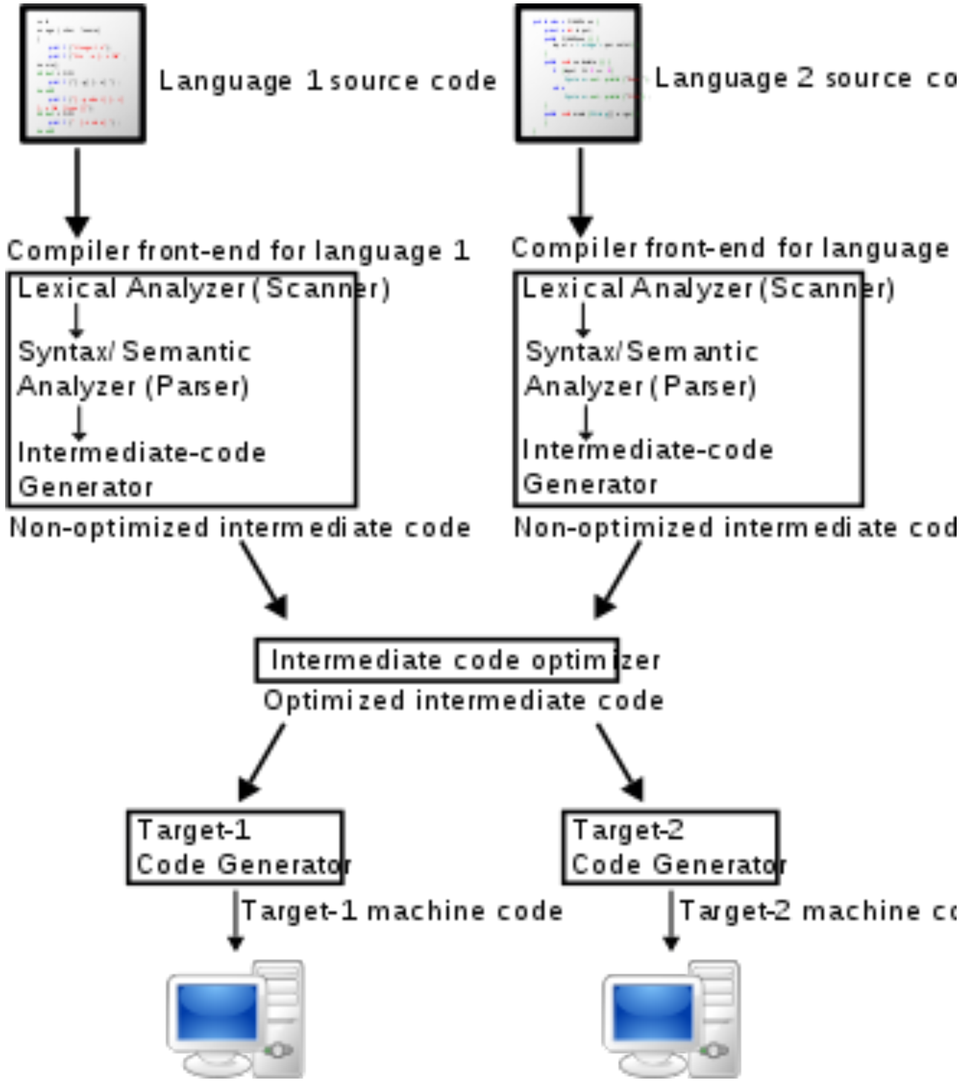
```
fib:
    mov edx, [esp+8]
    cmp edx, 0
    ja @f
    mov eax, 0
    ret

    @@:
    cmp edx, 2
    ja @f
    mov eax, 1
    ret

    @@:
    push ebx
    mov ebx, 1
    mov ecx, 1

    @@:
        lea eax, [ebx+ecx]
        cmp edx, 3
        jbe @f
        mov ebx, ecx
        mov ecx, eax
        dec edx
    jmp @b

    @@:
    pop ebx
    ret
```

Other programs makes it possible

- Three main implementation strategies:
- *Interpreter*: Runs any program in language Q
- *Compiler*: Converts a program in language Q to one in language L, for which you have a machine or interpreter
- *Hybrid*: A just-in-time (JIT) compiler compiles the program as it interprets it.

Compilers

Simpler view



A more detailed view

Reasoning (About Program Correctness)

Program Verification and Analysis

bugs, bugs everywhere

> A recent study found that 60-70% of vulnerabilities in iOS and macOS are caused by memory unsafety. Microsoft estimates that 70% of all vulnerabilities in their products over the last decade have been caused by memory unsafety. Google estimated that 90% of Android vulnerabilities are memory unsafety. An analysis of 0-days that were discovered being exploited in the wild found that more than 80% of the exploited vulnerabilities were due to memory unsafety.
>
> (Gaynor, 2019)

Why should we care about Correctness?

- When programs have errors, they fail, causing real damages.

Heartbleed: Hundreds of thousands of servers at risk from catastrophic bug

Code error means that websites can leak user details including passwords through 'heartbeat' function used to secure connections

Boeing Fixing New Software Bug on Max; Key Test Flight Nears

- Company says latest flaw shouldn't delay plane's return
- Issue involves aircraft's so-called 'trim' warning system

TECHNOLOGY

The "largest IT outage in history,"

We want our Software to be Correct!

Semantics: What does “correct” mean?

- To work with programs, we must know what they mean
  - *Semantics* comes from the Greek semaino, “to mean”.

$$x = a$$

Semantics in English: *The `=` operator takes a variable (like x) and a value (like a) and updates the value assigned to x.*

Semantics, more Formally

- *Semantics* : Defines meaning as a sequence of computational steps.
  - Stores: $S = \{x = 1, y = 2, \ldots xi = vi\}$
  - Operational semantics: $S \xrightarrow{e} S'$

$$\{xi = ai, \ldots\} \xrightarrow{x=b} \{xi = b, \ldots\}$$

- *Other Semantics*:
  - Denotational: Assigns mathematical objects (denotations) to program constructs
  - Axiomatic: Assign basic rules capturing the meaning. e.g. A map from Set to Set

Reasoning over Programs, using Semantics

```
{x = 1, y = 2}

x = x + 1;    {x = 2, y = 2}
y = y - 1;    {x = 2, y = 1}
x++;          {x = 3, y = 1}
x = x + y;    {x = 4, y = 1}
```

Err: If x >= 5

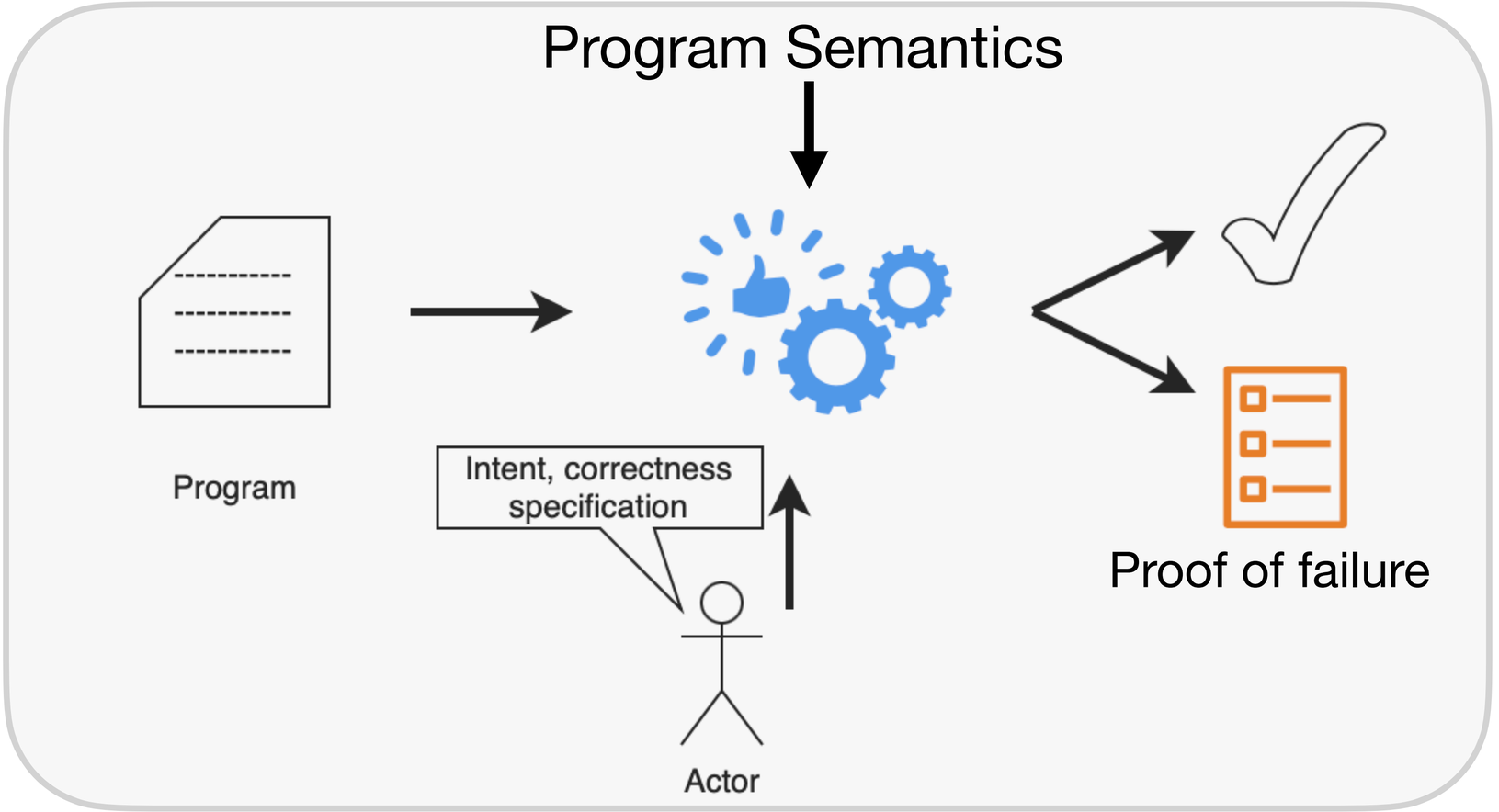
Will the above program lead to an Err

```
int main() {
    int x = 11;   {x = 11}
    int y = 20;   {x = 11, y = 20}
    int z = x / y; // Division
                  {x = 11, y = 20, z=0}
    printf("The result is: %d\n",
z);
    return 0;
}
```

Err: If z = 0

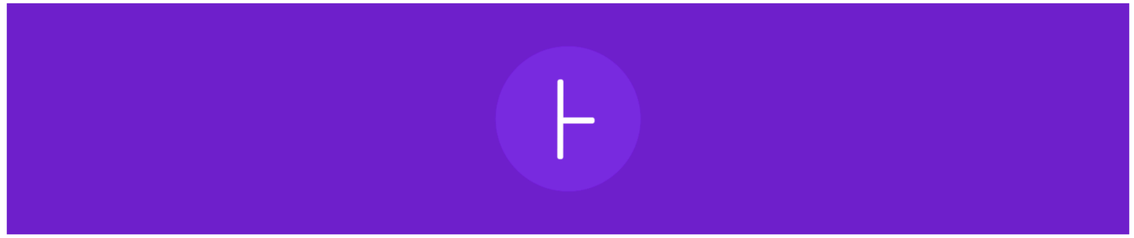
Will the above program lead to an Err

Program Verification : The general problem



Modern Program Verification Successes

Open-sourcing Facebook Infer: Identify bugs before you ship



Introducing s2n-tls, a New Open Source TLS Implementation

by Steve Schmidt | on 30 JUN 2015 | in Compliance | Permalink | Comments | Share

February 22, 2022: s2n has been renamed to s2n-tls. See details.

s2n-tls

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Inria

Awards & Honours

CompCert software program receives a prestigious award

Project Everest: Advancing the science of program proof

Published September 5, 2019

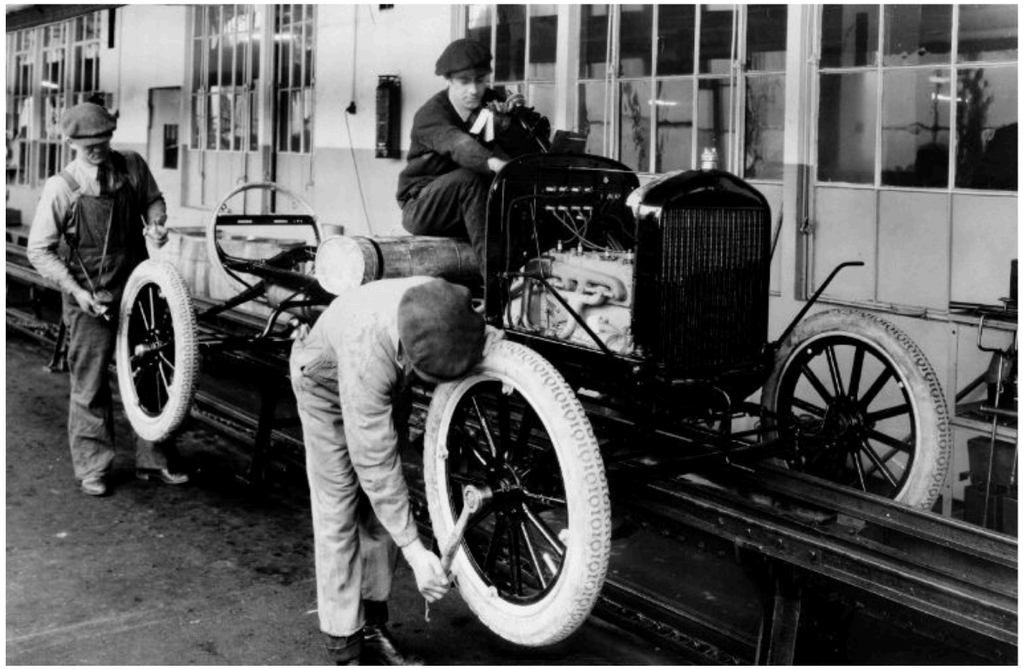
By Nikhil Swamy, Senior Principal Researcher

Share this page

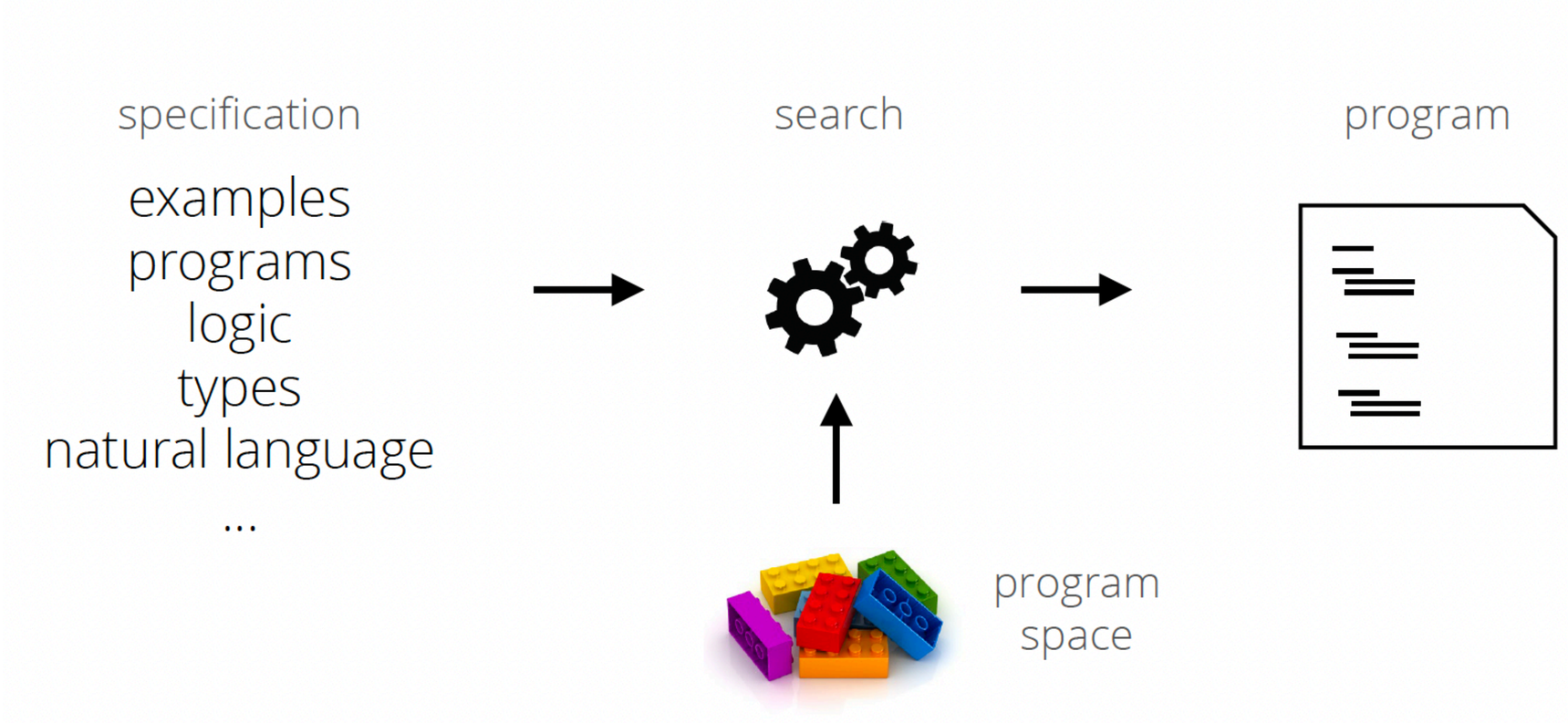
Synthesizing Programs

Program Synthesis

The Goal: Automate Programming



Program Synthesis



Notice the Duality with Program Verification

Program Synthesis Successes

FlashFill [Gulwani 2011]

FlashFill: Example

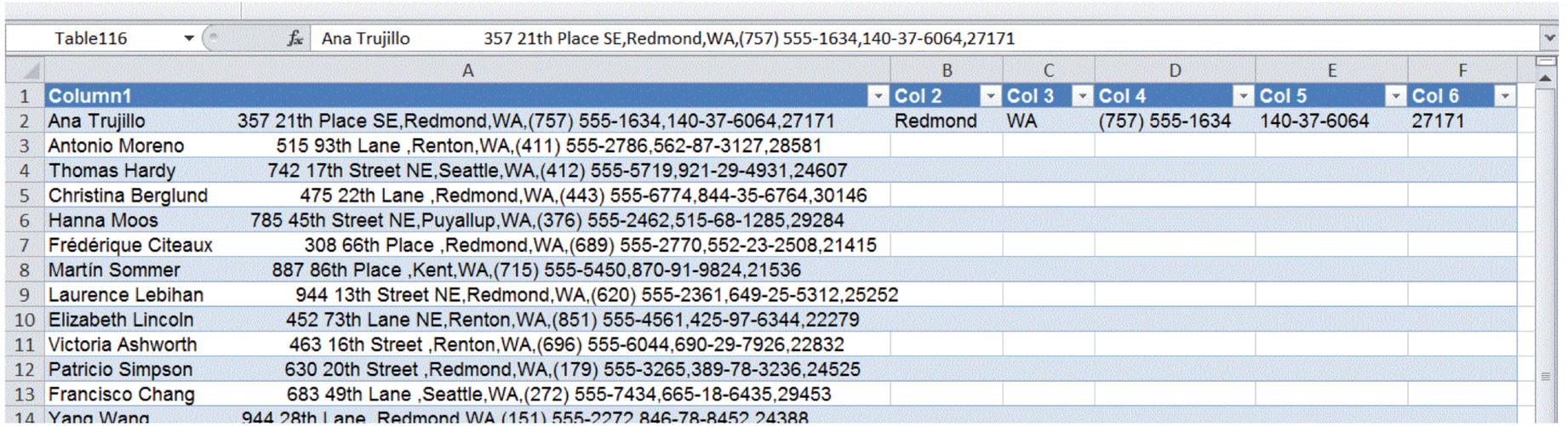


Table116 | Ana Trujillo | 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171

	A	B	C	D	E	F
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6
2	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171
3	Antonio Moreno 515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581					
4	Thomas Hardy 742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607					
5	Christina Berglund 475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146					
6	Hanna Moos 785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284					
7	Frédérique Citeaux 308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415					
8	Martín Sommer 887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536					
9	Laurence Lebihan 944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,25252					
10	Elizabeth Lincoln 452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279					
11	Victoria Ashworth 463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832					
12	Patricio Simpson 630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525					
13	Francisco Chang 683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453					
14	Yang Wang 944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388					

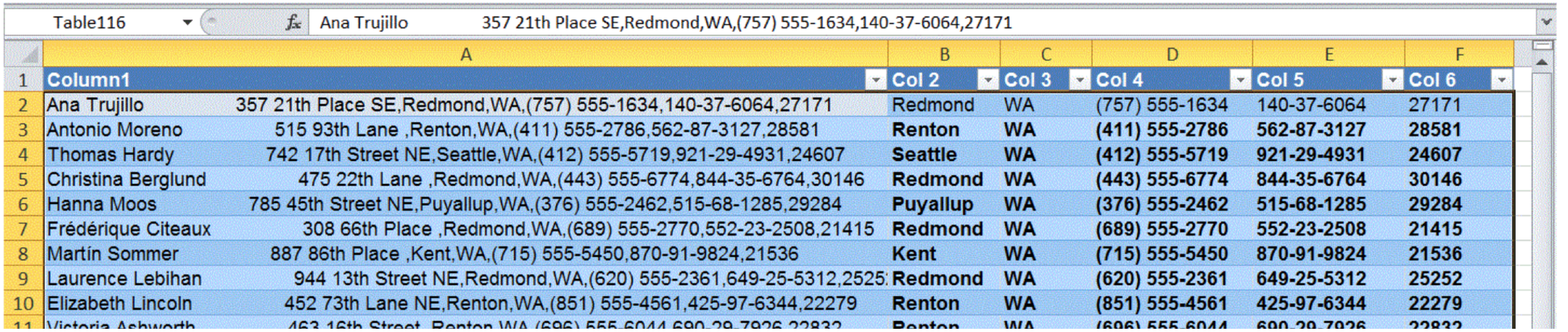


Table116 | Ana Trujillo | 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171

	A	B	C	D	E	F
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6
2	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171
3	Antonio Moreno 515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581	**Renton**	**WA**	**(411) 555-2786**	**562-87-3127**	**28581**
4	Thomas Hardy 742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607	**Seattle**	**WA**	**(412) 555-5719**	**921-29-4931**	**24607**
5	Christina Berglund 475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146	**Redmond**	**WA**	**(443) 555-6774**	**844-35-6764**	**30146**
6	Hanna Moos 785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284	**Puyallup**	**WA**	**(376) 555-2462**	**515-68-1285**	**29284**
7	Frédérique Citeaux 308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415	**Redmond**	**WA**	**(689) 555-2770**	**552-23-2508**	**21415**
8	Martín Sommer 887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536	**Kent**	**WA**	**(715) 555-5450**	**870-91-9824**	**21536**
9	Laurence Lebihan 944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,2525	**Redmond**	**WA**	**(620) 555-2361**	**649-25-5312**	**25252**
10	Elizabeth Lincoln 452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279	**Renton**	**WA**	**(851) 555-4561**	**425-97-6344**	**22279**
11	Victoria Ashworth 463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832	**Renton**	**WA**	**(696) 555-6044**	**690-29-7926**	**22832**

Under the hood

The Story of the Flash Fill Feature in Excel

by Sumit Gulwani on Sep 14, 2021 | Tags: Flash Fill, MIP award, program synthesis, Programming by Examples

Automating String Processing in Spreadsheets Using Input-Output Examples

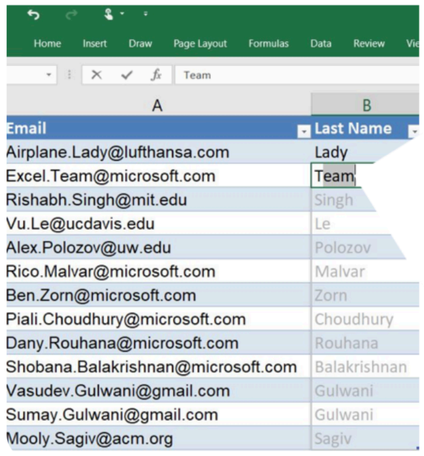
Sumit Gulwani
Microsoft Research, Redmond, WA, USA
sumitg@microsoft.com

Abstract

We describe the design of a string programming/expression language that supports restricted forms of regular expressions, conditionals and loops. The language is expressive enough to represent a wide variety of string manipulation tasks that end-users struggle with. We describe an algorithm based on several novel concepts for

their task [9]. More significantly, programmi
perform tedious and repetitive tasks such as
like names/phone-numbers/dates from one fo
cleansing, extracting data from several text fil
a single document, etc. Spreadsheet systems
allow users to write macros using a rich inbuil
numerical functions, or to write arbitrary scri

Excel 2013's coolest new feature that
hould have been available years ago"



Automating String Processing in Spreadsheets Using Input-Output Examples

Sumit Gulwani
Microsoft Research, Redmond, WA, USA
sumitg@microsoft.com

Abstract

We describe the design of a string programming/expression language that supports restricted forms of regular expressions, conditionals and loops. The language is expressive enough to represent a wide variety of string manipulation tasks that end-users struggle with. We describe an algorithm based on several novel concepts for synthesizing a desired program in this language from input-output examples. The synthesis algorithm is very efficient taking a fraction of a second for various benchmark examples. The synthesis algorithm is interactive and has several desirable features: it can rank multiple solutions and has fast convergence, it can detect noise in the user input, and it supports an active interaction model wherein the user is prompted to provide outputs on inputs that may have multiple computational interpretations.

their task [9]. More significantly, programming is still required to perform tedious and repetitive tasks such as transforming entities like names/phone-numbers/dates from one format to another, data cleansing, extracting data from several text files or web pages into a single document, etc. Spreadsheet systems like Microsoft Excel allow users to write macros using a rich inbuilt library of string and numerical functions, or to write arbitrary scripts using a variety of programming languages like Visual Basic, c# .Net. Since end-users are not proficient in programming, they find it too difficult to write desired macros or scripts.

We have performed an extensive case study of spreadsheet help forums and identified that string processing is one of the most common class of programming problems that end-users struggle with. This is not surprising given that languages like Perl,

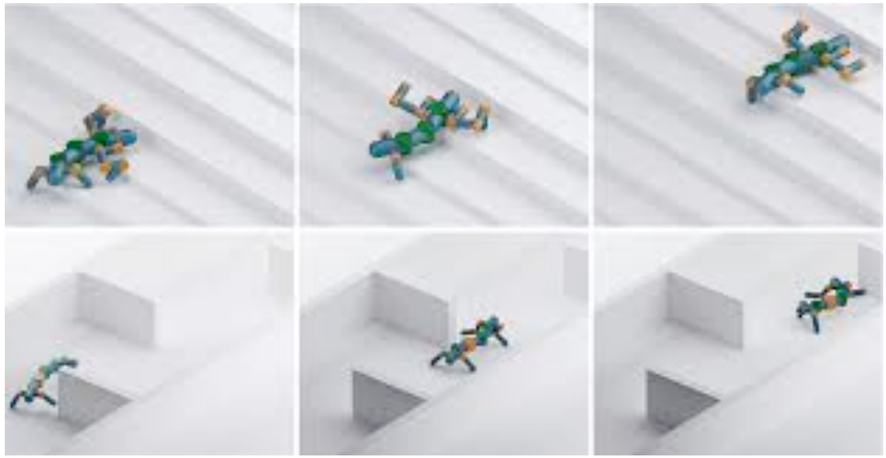
FlashFill++: Scaling Programming by Example by Cutting to the Chase

JOSÉ CAMBRONERO*, Microsoft, USA
SUMIT GULWANI*, Microsoft, USA
VU LE*, Microsoft, USA

Major Idea is VSAs

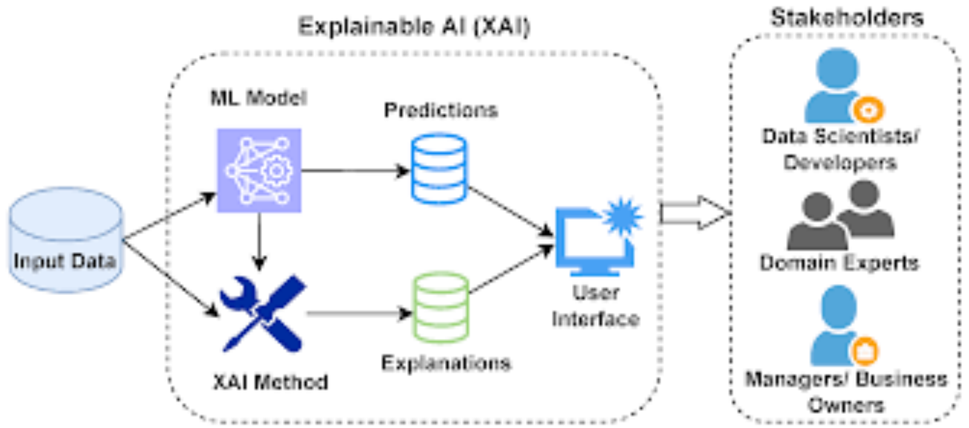
Other Program Synthesis Successes

Robotics design and path planning



Data Migration

Explainable AI (XAI)



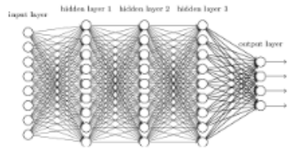
Clement, T. et. al. 2023

Neurosymbolic Programming

Ingredients of ML

$$S = \{(x_i, y_i)\}_{i=1}^{N}$$

Training Data



Model Class(es)

$$L(a,b) = (a-b)^2$$

Loss Function

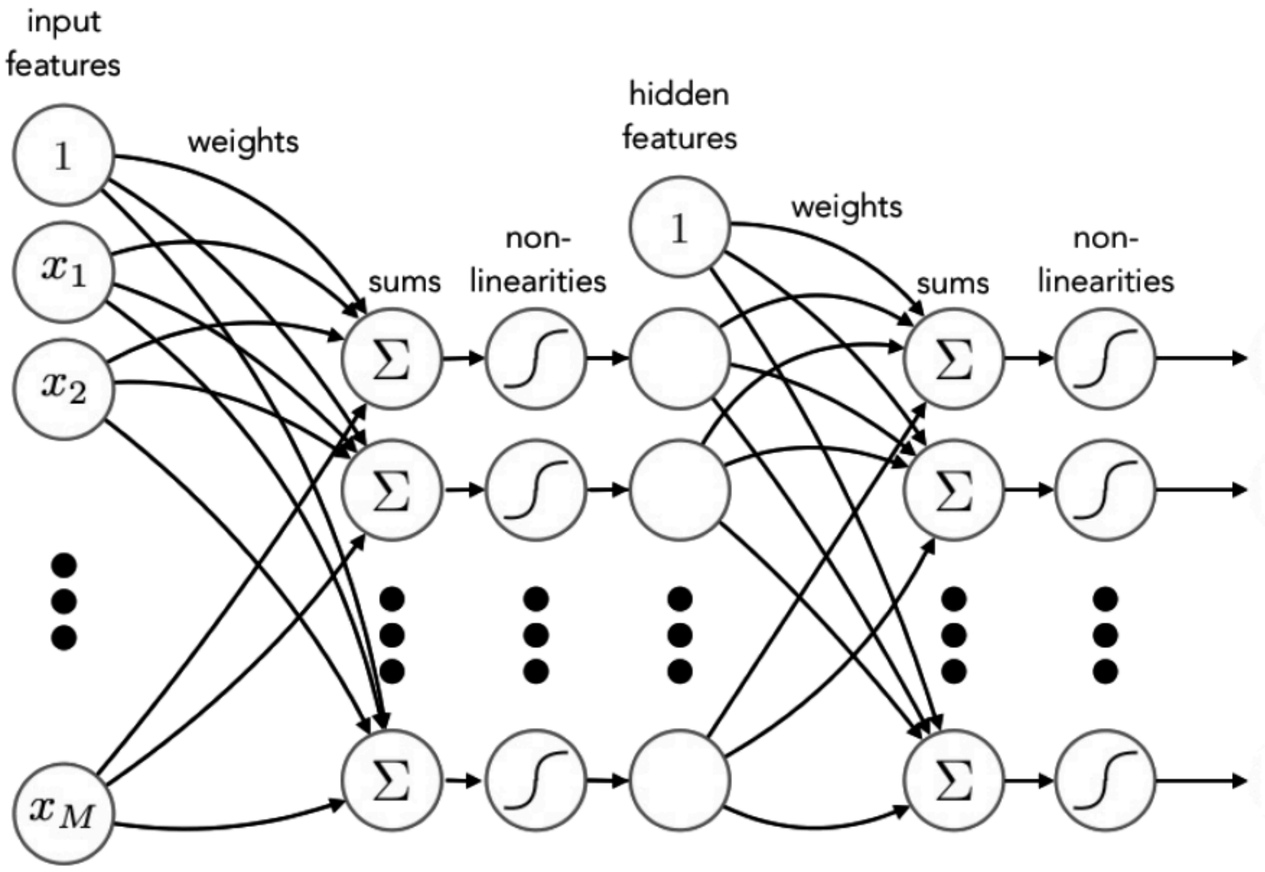
$$\underset{w,b}{\operatorname{argmin}} \sum_{i=1}^{N} L\big(y_i, f(x_i \mid w,b)\big)$$

Learning (Optimization)

Profit!

Deep NN

-



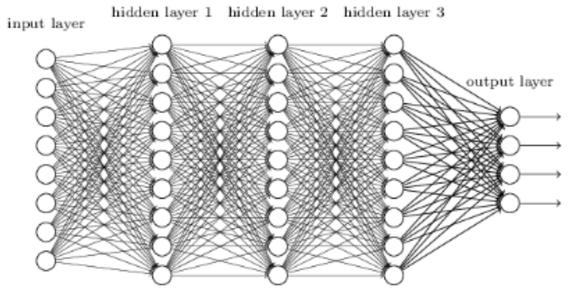
Key Benefits:

- Expressiveness
- Differentiable Learning

•••

Slide by Joe Marino

Weaknesses of Deep Learning



Uninterpretable → Hard to trust/control

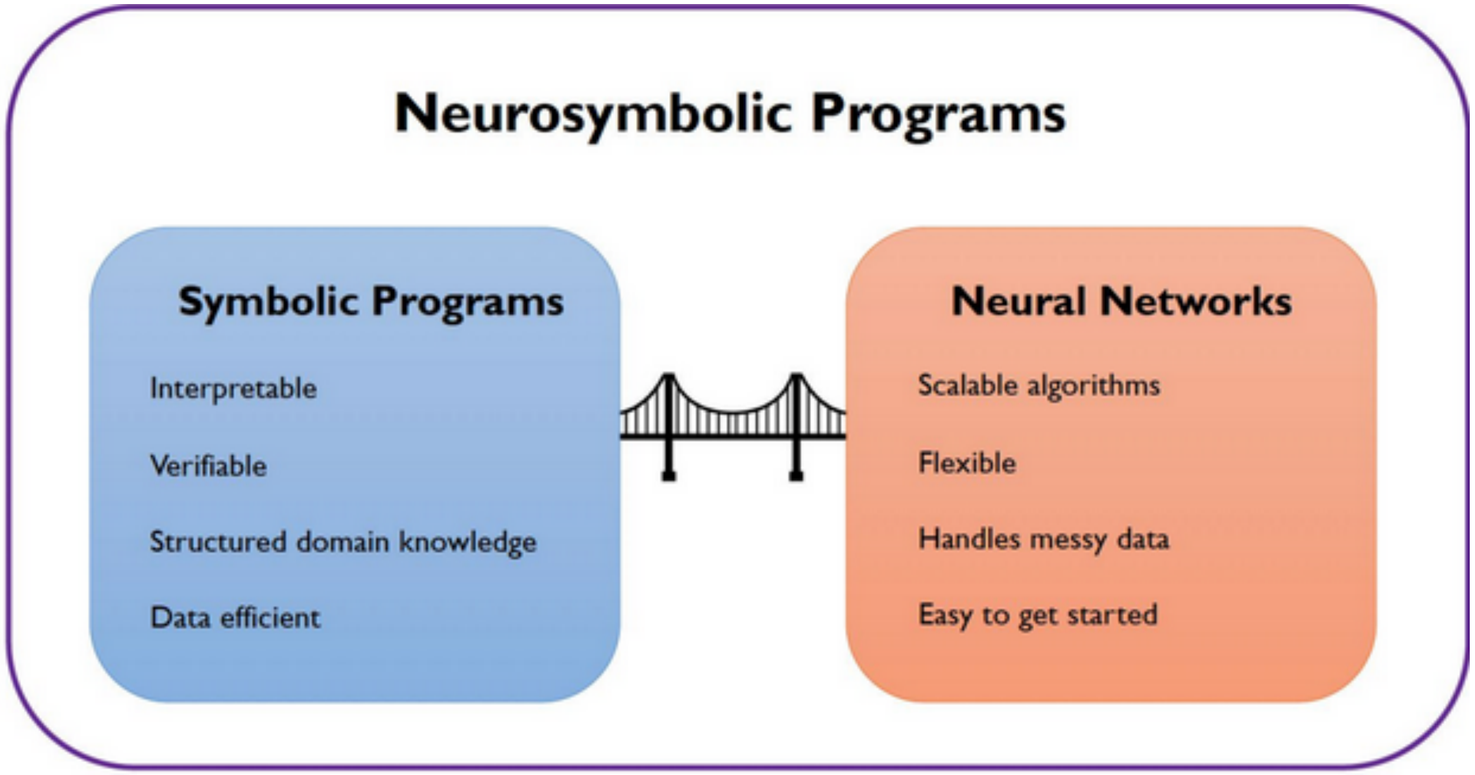
Lack of domain knowledge →
Unreliable training, high sample complexity

Opaque inductive bias → Brittle model

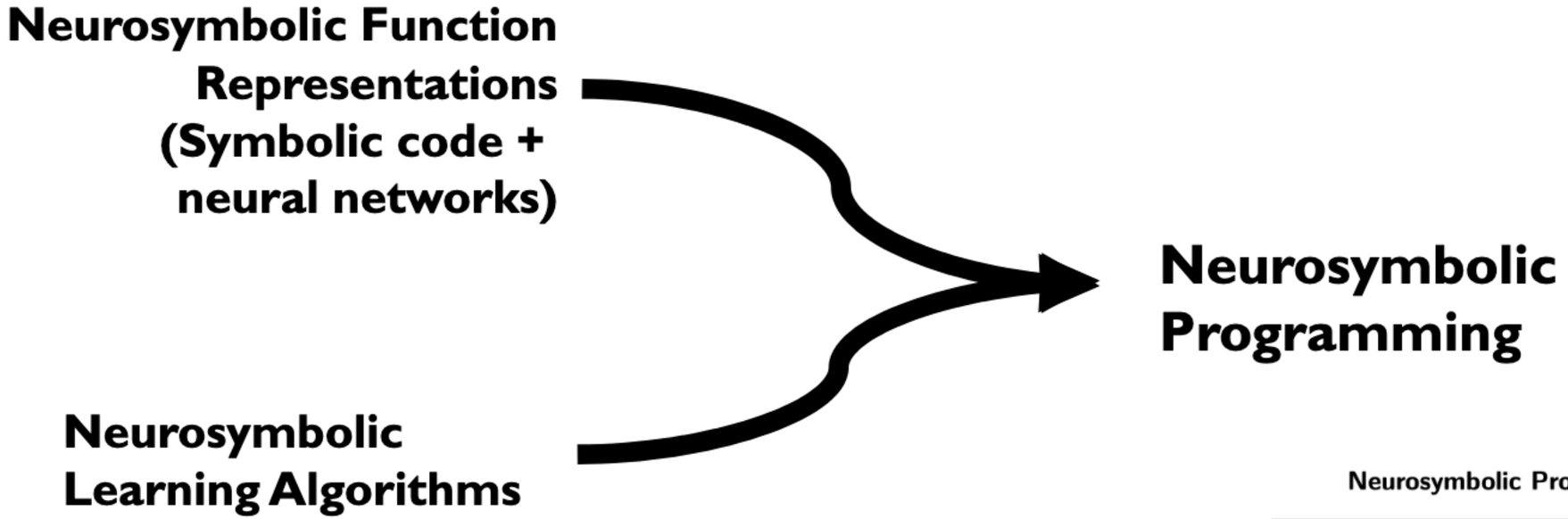
Key Insight: Use Symbolic Knowledge

A.k.a. "Code" or "Programs"

Neuorosymbolic Programs



Neuosymbolic Programming



Neurosymbolic Programming. Chaudhuri, Ellis, Polozov, Singh, Solar-Lezama, Yue. Foundations and Trends in Programming Languages, 2021.

Neurosymbolic Programming

Swarat Chaudhuri
UT Austin
swarat@cs.utexas.edu

Kevin Ellis
Cornell
kellis@cornell.edu

Oleksandr Polozov
Google
polozov@google.com

Rishabh Singh
Google
rising@google.com

Armando Solar-Lezama
MIT
asolar@csail.mit.edu

Yisong Yue
Caltech
yyue@caltech.edu

Differentiable Programming.

Traditional Programs vs ML

- Programming:
  - The programmer specifies exact rules for how input maps to output.
  - Example: Sorting algorithm.

```
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(len(arr) - i - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

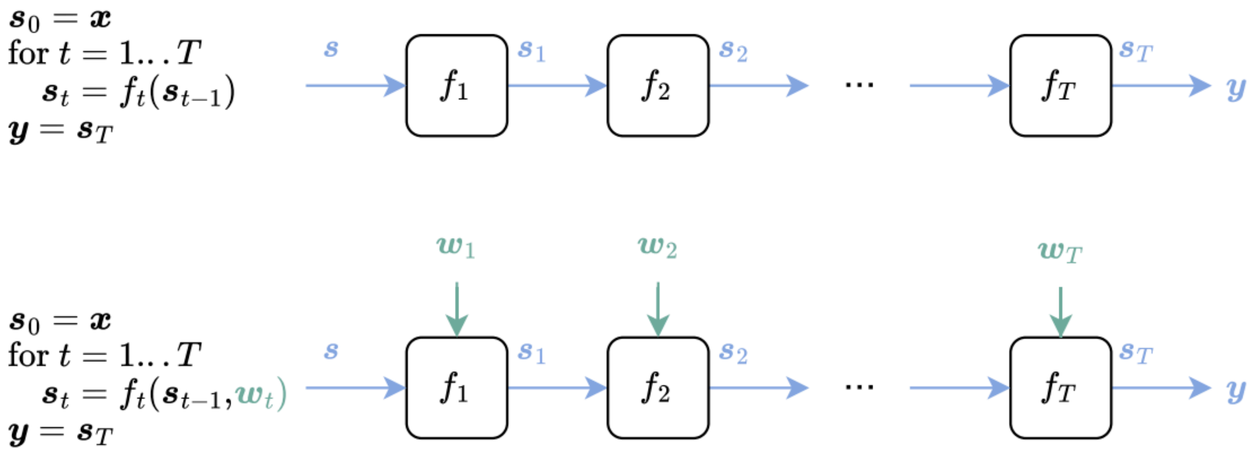
  - Completely deterministic, no “learning” involved.

Traditional Programs vs ML

- Machine Learning:
  - The programmer specifies a model family (e.g., linear regression, neural net) and a loss function.
  - The parameters are optimized automatically from data using end-to-end training process.
    - Learn both feature extraction and task execution simultaneously.
  - Example: Linear regression: $y = w \cdot x + b$

What is Diff Programming?

- A programming paradigm in which complex computer programs (including those with control flows and data structures) can be differentiated end-to-end automatically.
- This enables gradient-based optimization of parameters in the program.
- Computation Graphs like traditional programs, though parameterized.



What is Diff Programming?

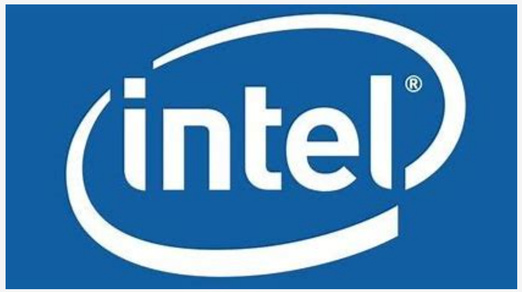
- History:
  - Probably: Proposed and coined by Colah using the connection b/w DL and Functional Programming.
- Includes Two related things:
  - Differentiating through programs-
    - Using Numerical Diff or Symbolic Diff or Automatic Diff.
  - Functions in the programs should have well defined derivatives-
    - Designing meaningful differentiable programs.
    - principled differentiable operations.

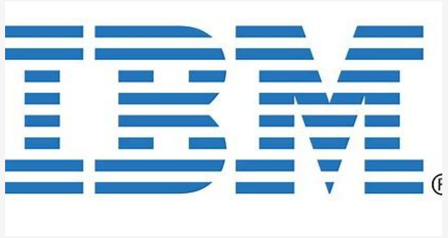
A Glimpse of PL/C Research @IITH

People + Areas

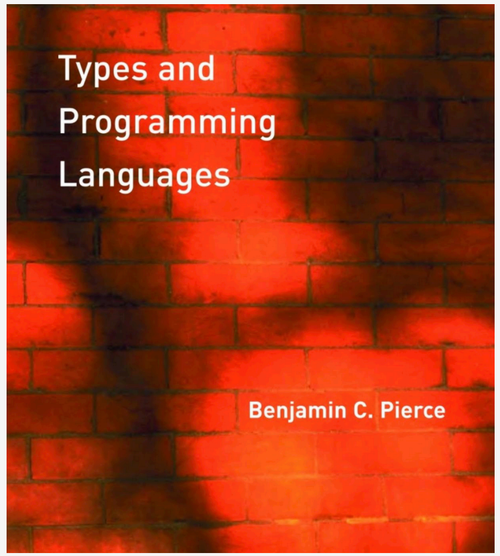
01	Program Analysis and Compilers and Synthesis using Machine Learning
02	Polyhedral Compilation,Compiler Optimization Techniques for CPUs & GPUs
03	Data Race Checking & Parallel Code Compliance Standards
04	Neurosymbolic Programming
05	Novel Type Systems for Incorrectness

Collaborating Industries: Another Reason to Join PL/C





Some Resources:



OREGON PROGRAMMING LANGUAGES SUMMER SCHOOL

Each year, the ACM Sigplan Oregon Programming Language Summer School covers a topic in the foundation of programming languages. The summer school is intended primarily for graduate students, but has been attended by undergrads, professors and industry professionals as well. Sponsors have included ACM Sigplan, BAE Systems, The Charles Stark Draper Laboratory, Inc., Facebook, Galois, Google, Indiana University, INRIA, Intel, Jane Street Capital, Microsoft Research, the National Science Foundation, The US Air Force, and Univeristy of Oregon.

Current Year
OPLSS 2002 — Proofs as Programs
OPLSS 2003 — Foundations of Security
OPLSS 2004 — Software Security
OPLSS 2005 — Reliable Computing
OPLSS 2006 — Concurrent and Distributed Software
OPLSS 2007 — Integrating with the External World
OPLSS 2008 — Logic and Theorem Proving
OPLSS 2009 — Theory & Practice of Language Implementation
OPLSS 2010 — Logic, Languages, Compilation, and Verification
OPLSS 2011 — Types, Semantics and Verification
OPLSS 2012 — Logic, Languages, Compilation, and Verification
OPLSS 2013 — Types Logic, and Verification
OPLSS 2014 — Types, Logic, Sematics, and Verification
OPLSS 2015 — Types, Logic, Sematics, and Verification
OPLSS 2016 — Types, Logic, Sematics, and Verification
OPLSS 2017 — A Spectrum of Types
OPLSS 2018 — Parallelism and Concurrency
OPLSS 2019 — Foundations of Probabilistic Programming and Security
OPLSS 2021 — Foundations of Programming and Security
OPLSS 2022 — Types, Semantics, and Program Reasoning
OPLSS 2023 — Types, Semantics, and Logic

The Programming Languages Enthusiast

ABOUT THE PL ENTHUSIAST

PL/C @IITH Thanks for listening!

01	Program Analysis and Compilers using Machine Learning
02	Polyhedral Compilation,Compiler Optimization Techniques for CPUs & GPUs
03	Data Race Checking & Parallel Code Compliance Standards
04	Scalable Program Synthesis from Libraries
05	Novel Type Systems for Incorrectness